

Customer Churn Analytics using Amazon Redshift

Solution Design Document (SDD)

Project Name: Customer Churn Analytics using Amazon Redshift

Prepared By : Shreyash Singh

Technology Stack

- Amazon S3
 - Amazon Redshift
 - Python
 - Psycopg2
 - AWS IAM
 - SQL
 - Redshift Query Editor
 - Git & GitHub
-

1. Executive Summary

The Customer Churn Analytics System is designed to ingest, process, store, and analyze telecom customer churn data using Amazon Redshift as the central data warehouse.

The solution follows a modern data warehousing approach where raw customer data is stored in Amazon S3 and loaded into Redshift using the COPY command. The data is processed through staging tables and transformed into an analytical table optimized for reporting and business intelligence.

The system enables organizations to identify churn trends, analyze customer behavior, estimate revenue loss, and support data-driven retention strategies.

Key capabilities include:

- Automated data loading from Amazon S3
 - Data warehousing using Amazon Redshift
 - ETL processing using Python and SQL
 - Customer churn analysis
 - Revenue loss estimation
 - Population-based customer analysis
 - Query optimization using DISTKEY and SORTKEY
-

2. Business Problem

Telecommunication companies face significant revenue loss due to customer churn.

Challenges include:

- Large volumes of customer data
- Difficulty identifying churn patterns
- Lack of centralized analytical reporting
- Slow query performance on traditional databases
- Limited visibility into revenue impact

The proposed solution leverages Amazon Redshift to build a scalable analytical platform capable of processing large datasets efficiently.

3. Objectives

The system aims to:

- ✓ Load customer churn datasets from Amazon S3
 - ✓ Store data in Amazon Redshift
 - ✓ Create staging and analytical layers
 - ✓ Analyze customer churn behavior
 - ✓ Identify cities with high churn rates
 - ✓ Measure revenue loss due to churn
 - ✓ Perform ZIP code population analysis
 - ✓ Optimize query performance using Redshift best practices
-

4. Solution Architecture

Data Flow :

1. Customer churn dataset uploaded to Amazon S3
2. Redshift COPY command loads data into Raw Staging Tables
3. Data transformed into Staging Tables
4. Analytical Table created using joins and transformations
5. SQL queries generate business insights
6. Results displayed through Python scripts using Psycopg2

Architecture Components

- **Amazon S3**
 - Stores source datasets
- **Amazon Redshift**

- Data warehouse platform
- **Python + Psycopg2**
 - ETL execution and analytics
- **IAM Role**
 - Secure Redshift access to S3

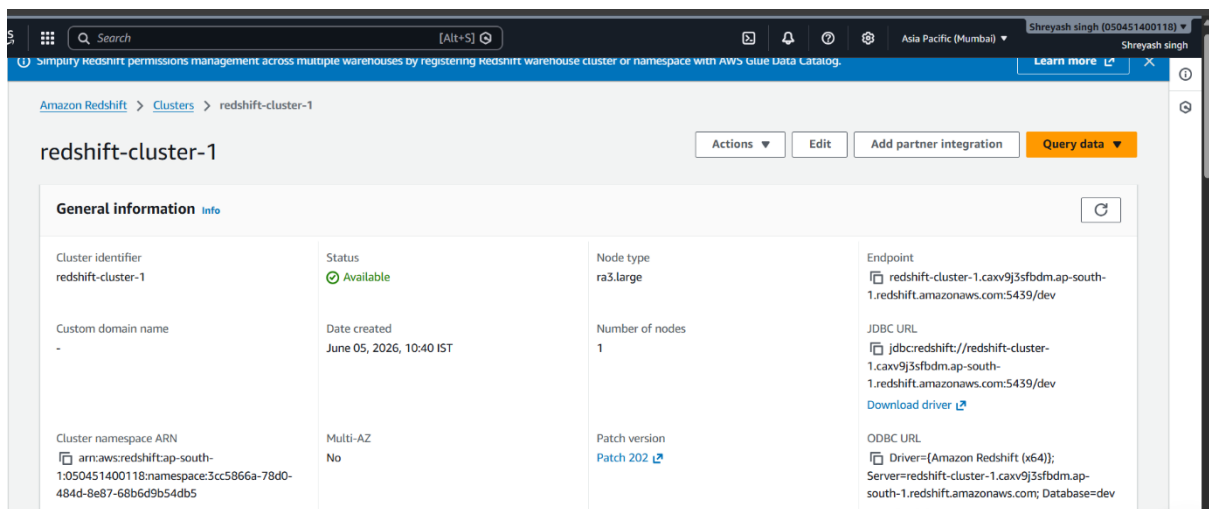
5. Data Loading Process

Customer data is loaded from Amazon S3 using the Redshift COPY command.

Best practices followed:

- CSV format
- Ignore file headers
- Correct delimiter handling
- Empty value handling
- Staging table loading

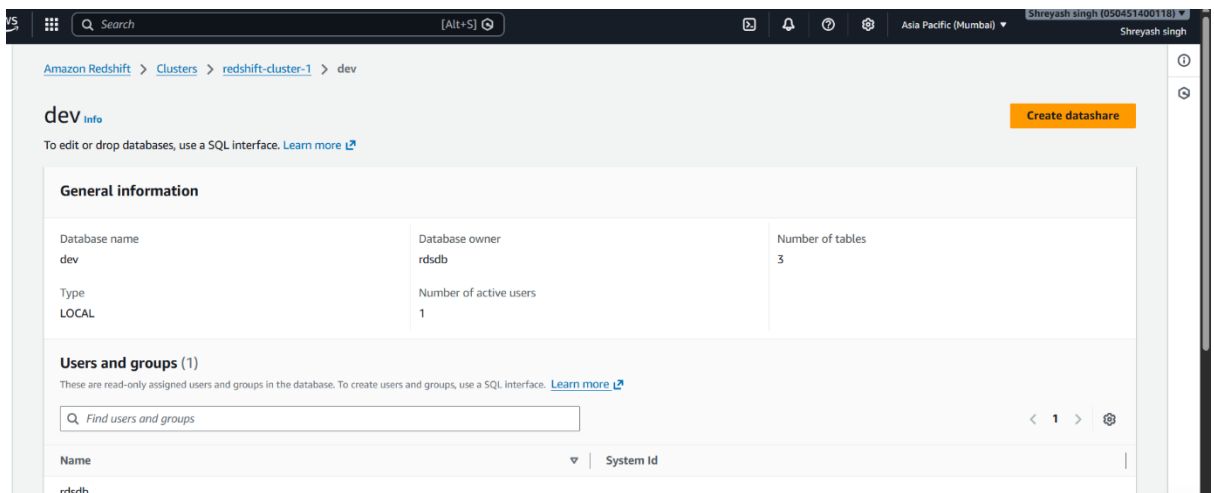
6. AMAZON REDSHIFT



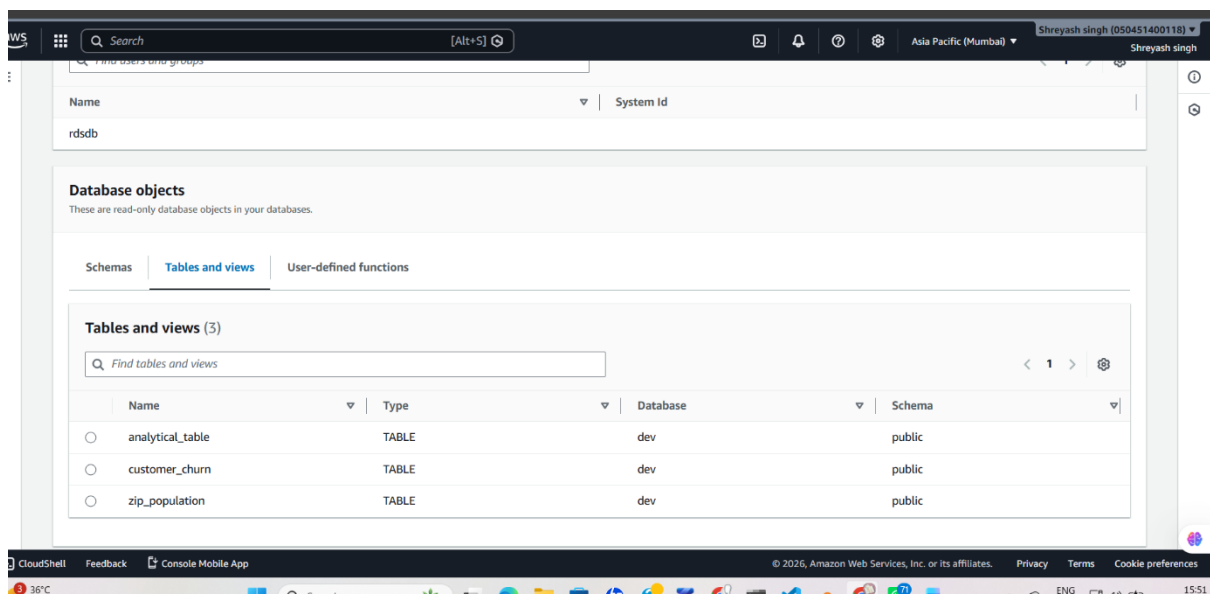
The screenshot shows the Amazon Redshift console interface. At the top, there's a navigation bar with the Amazon Redshift logo and a search bar. Below the navigation bar, the breadcrumb trail indicates the current location: Amazon Redshift > Clusters > redshift-cluster-1. The main content area displays the details for the cluster 'redshift-cluster-1'. On the right side of the cluster name, there are buttons for 'Actions', 'Edit', 'Add partner integration', and 'Query data'. Below this, there's a 'General information' section with a refresh icon. The information is presented in a table-like format with four columns: Cluster identifier, Status, Node type, and Endpoint. The 'Cluster identifier' is 'redshift-cluster-1'. The 'Status' is 'Available' with a green checkmark icon. The 'Node type' is 'ra3.large'. The 'Endpoint' is 'redshift-cluster-1.caxv9j3sfbdm.ap-south-1.redshift.amazonaws.com:5439/dev'. Other details include 'Custom domain name' (empty), 'Date created' (June 05, 2026, 10:40 IST), 'Number of nodes' (1), 'Cluster namespace ARN' (arn:aws:redshift:ap-south-1:050451400118:namespace:3cc5866a-78d0-484d-8e87-68b6d9b54db5), 'Multi-AZ' (No), 'Patch version' (Patch 202), 'JDBC URL' (jdbc:redshift://redshift-cluster-1.caxv9j3sfbdm.ap-south-1.redshift.amazonaws.com:5439/dev), and 'ODBC URL' (Driver={Amazon Redshift (x64)}; Server=redshift-cluster-1.caxv9j3sfbdm.ap-south-1.redshift.amazonaws.com; Database=dev).

Cluster identifier	Status	Node type	Endpoint
redshift-cluster-1	Available	ra3.large	redshift-cluster-1.caxv9j3sfbdm.ap-south-1.redshift.amazonaws.com:5439/dev
Custom domain name	Date created	Number of nodes	JDBC URL
-	June 05, 2026, 10:40 IST	1	jdbc:redshift://redshift-cluster-1.caxv9j3sfbdm.ap-south-1.redshift.amazonaws.com:5439/dev
Cluster namespace ARN	Multi-AZ	Patch version	ODBC URL
arn:aws:redshift:ap-south-1:050451400118:namespace:3cc5866a-78d0-484d-8e87-68b6d9b54db5	No	Patch 202	Driver={Amazon Redshift (x64)}; Server=redshift-cluster-1.caxv9j3sfbdm.ap-south-1.redshift.amazonaws.com; Database=dev

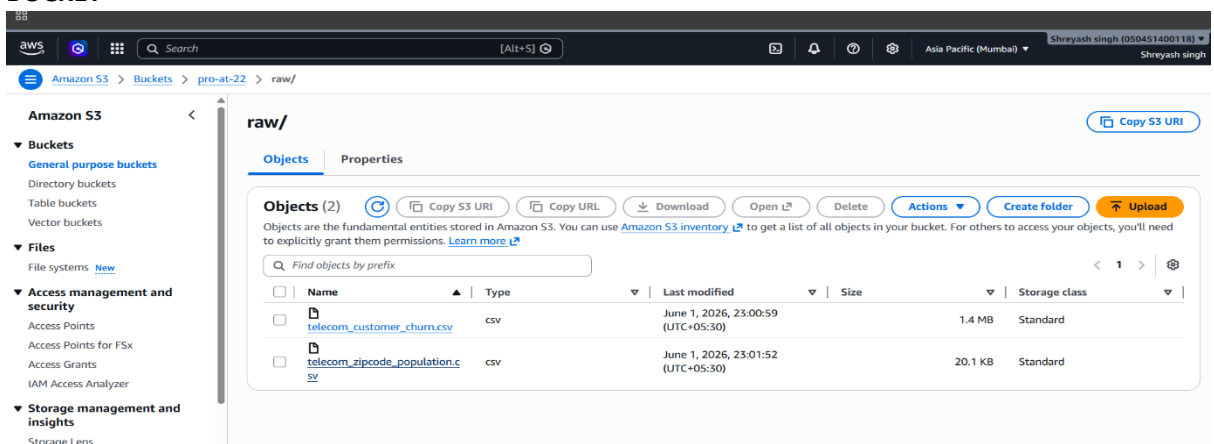
7. DATABASE



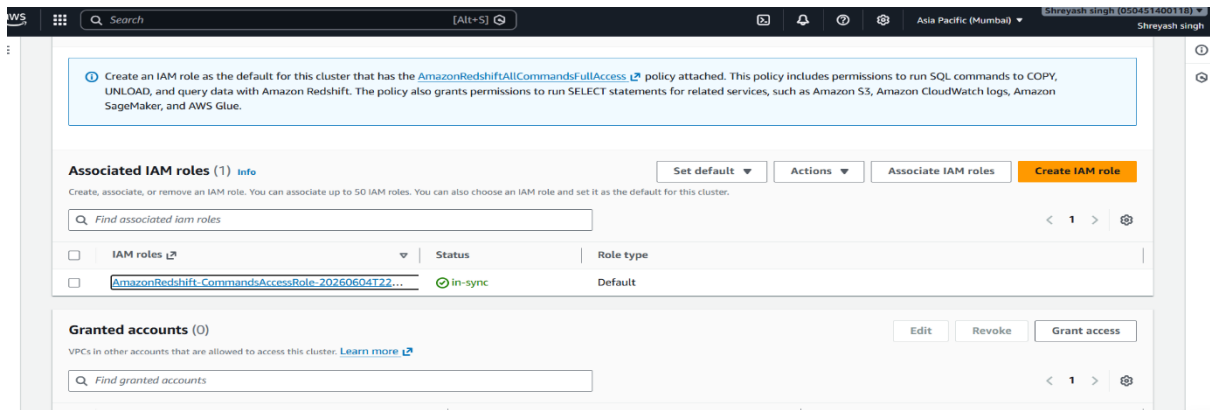
8.TABLES



8. BUCKET



9.IAM



10.ARCHITECTURE

